

# git 기초 명령

## 학습목표

git 기초 개념과 명령을 학습한다

## 목차

|                                         |    |
|-----------------------------------------|----|
| 1. git의 기본 .....                        | 2  |
| 1) 시작하기 .....                           | 2  |
| 2) git repository.....                  | 3  |
| 3) 변경을 기록하는 commit.....                 | 3  |
| 4) working tree와 staging area.....      | 4  |
| 5) working tree 내부의 파일의 상태 .....        | 4  |
| 6) working tree 내부의 파일의 상태 변화.....      | 5  |
| 2. git 기초 명령 .....                      | 6  |
| 1) local repository 명령 .....            | 6  |
| 2) staging area 등록/취소 명령 .....          | 6  |
| 3) commit 명령.....                       | 7  |
| 3. 실습 .....                             | 8  |
| 1) 프로젝트 폴더 만들기.....                     | 11 |
| 2) local repository 만들기 (git init)..... | 12 |
| 3) 소스코드 파일 만들기.....                     | 15 |
| 4) 파일 상태 확인 (git status).....           | 17 |
| 5) 새 파일을 tracked 상태로 바꾸기 (git add)..... | 18 |
| 6) 수정 내용 등록하기 (git commit).....         | 19 |
| 7) 커밋 목록 보기 (git log).....              | 21 |
| 4. remote repository 명령.....            | 22 |
| 1) remote repository에 연결.....           | 22 |
| 2) remote repository에 업로드.....          | 22 |
| 3) remote repository에 다운로드.....         | 22 |
| 5. 실습 .....                             | 23 |
| 1) github에 계정 생성.....                   | 23 |
| 2) remote repository 생성.....            | 23 |
| 3) remote repository에 연결.....           | 25 |
| 4) 작업 내용 업로드.....                       | 26 |
| 5) remote repository 다운로드.....          | 28 |

# 1. git의 기본

## 1) 시작하기

### 소스코드 파일의 버전 백업

여러분은 소스코드를 수정하다 망해버려서, 수정 전 상태로 되돌리고 싶을 때 어떻게 하는가?

가장 단순하고 확실한 방법은, 수정을 시작하기 전에 소스코드 파일들을 따로 백업해 두는 것이다. 수정하다 망해버린 파일들을 삭제하고, 백업해둔 파일을 다시 꺼내면 간단하게 수정 전 상태로 되돌아 갈 수 있다.

그런데 이 방법의 문제는, 소스코드 파일을 편집할 때마다 매번 백업하는 일이 번거롭다는 점이다. 그래서 가끔 백업을 깜빡했다가 낭패를 보곤 한다.

또 백업이 너무 많아지면, 백업의 구체적인 내용이 기억나지 않아서 혼란스럽기도 하다.

### 프로젝트 공동 작업

친구들이랑 같이 프로젝트를 할 때, 내가 애써서 구현한 코드를 다른 팀원이 실수로 망쳐버린 경우는 없었나?

팀원들이 각자 구현한 버전들을 전부 취합할 때 어려움을 겪은 적은 없나?

### 버전 관리 시스템

위와 같은 문제들을 해결하기 위해 만들어진 것이 git과 같은 버전 관리 시스템이다.

git에서는 소스 코드가 변경된 이력을 쉽게 확인할 수 있고, 특정 시점에 저장된 버전과 비교하거나 특정 시점의 버전으로 되돌아갈 수도 있다.

또 내가 팀 공동 repository에 올리려는 파일이 팀원 누군가 편집한 내용과 충돌한다면, 공동 repository에 올릴 때 경고 메시지가 발생하고 업로드는 취소된다. 그래서 누군가 애써 편집한 내용을 날려버리는 실수를 이제 걱정하지 않아도 된다.

## 2) git repository

git repository에는 소스코드 파일의 수정 과정에서 만들어졌던 전체 버전들이 전부 저장된다.

### remote repository와 local repository

git repository는 local repository와 remote repository가 있다.

#### remote repository

팀원들이 공유하기 위한 repository다. 팀원들 모두 접근할 수 있는 remote 서버에 만들어진다. 무료로 이용할 수 있는 대표적인 git remote repository 서버는 github 이다.

#### local repository

개발자 개인의 PC에 생성되는 개인 전용 repository입니다.

평소에는 팀원 각자 개인 PC의 local repository에 버전을 저장하며 작업하다가, 그동안 작업한 내용을 팀원들에게 공개하고 싶을 때, local repository에 저장된 내용을 remote repository에 업로드 한다.

누군가 remote repository에 작업한 내용을 업로드하면, 다른 팀원들은 그 내용을 각자의 개인 PC의 local repository에 다운로드할 수 있다.

### local repository 만들기

개인 PC에 local repository를 만드는 방법은 두 가지가 있다.

(1) 비어있는 local repository를 새로 만든다

(2) 이미 만들어져 있는 remote repository를 다운로드해서 local repository를 새로 만든다.

## 3) 변경을 기록하는 commit

local repository에 새 버전을 등록하는 작업을 commit이라고 한다. commit을 할 때에만 local repository에 버전이 등록된다.

어느정도 의미있는 소스 코드 수정 작업을 마쳤을 때마다, 그 작업을 마친 버전을 local repository에 commit하는 것이 바람직하다.

예를 들어, 어떤 기능 구현을 완료했다거나, 어떤 버그를 해결했다면, 그 작업 결과 버전을 local repository에 commit하자. commit은 소스코드 변경 이력을 저장소에 기록하는 중요한 작업이다.

### commit 메시지

local repository에 commit을 등록할 때, commit 메시지를 필수로 입력해야 한다. 작업 내용을 commit 메시지에 입력하자. 작업 내용을 팀원들이 쉽게 이해할 수 있도록 입력하자.

commit 메시지가 여러 줄인 경우에는 다음과 같은 형식을 따르자.  
첫째 줄에 commit 메시지 제목을 적고  
둘째 줄은 비워두고  
셋째 줄부터 commit 메시지 본문을 적는다.

## commit hash

각 commit에는 알파벳과 숫자로 이루어진 40자리의 hash가 자동으로 부여된다.  
commit hash는 그 commit을 식별하기 위한 ID 이다. 즉 commit ID 이다.

## 4) working tree와 staging area

git 버전 관리 대상인 소스코드 폴더를 working tree라고 부른다. (working directory)

commit을 실행하기 전 local repository와 working tree 사이에 staging area 라고 불리는 가상의 공간이 있다.

git의 commit 작업은 working tree에 있는 변경 내용을 전부 repository에 바로 기록하는 것이 아니라, staging area에 등록된 파일들의 변경 내용만 repository에 기록한다.

따라서 어떤 파일의 변경 사항을 repository에 기록하기 위해서는, commit하기 전에 먼저 그 파일을 staging area에 등록해야 한다. staging area에 등록된 파일만 다음 번 commit에서 기록된다.

예를 들어, 10개의 파일을 수정했지만 그 중에 7개 파일의 변경 사항만 repository에 기록하고 싶다면, 그 7개 파일을 staging area에 등록한 후 commit 해야 한다.

이렇게 staging area에 등록된 파일의 변경 사항만 repository에 기록되기 때문에, working tree에 있는 파일들 중 내가 원하는 변경 사항만 repository에 commit할 수 있다.

staging area를 index 라고 부르기도 한다.

### working tree 스냅샷(snapshot)

commit을 하면, working tree의 현재 상태가 local repository에 기록된다.  
그래서 나중에 그 스냅샷 상태로 working tree를 되돌리는 것이 가능하다.  
단 staging area에 등록된 파일들만 local repository에 기록된다.

## 5) working tree 내부의 파일의 상태

working tree 내부의 파일의 상태는 다음 상태 중 하나이다.

### untracked

한 번도 commit 되지 않았고, 아직 staging area에 등록되지도 않은 새 파일의 상태는 untracked 이다.

### staged

staged 상태의 파일은 다음 두 경우 중 하나이다.

- (1) 아직 한 번도 commit되지 않은 새 파일이고, 처음으로 commit되기 위해, staging area에 등록된 파일의 상태는 staged 이다. (untracked => staged) new file
- (2) 변경 내용이 commit된 이후 수정되었고, 그 수정 내용이 다시 commit 되기 위해, staging area에 등록된 파일의 상태는 staged 이다. (modified => staged)

### unmodified

변경 내용이 commit된 이후로 수정되지 않은 파일들의 상태는 unmodified 이다.

### modified

변경 내용이 commit된 이후 수정되었지만, staging area에 다시 등록되지 않아서, 다음 commit 대상이 아닌 파일의 상태는 dirty / modified 이다.

## 6) working tree 내부의 파일의 상태 변화

### 상태 변화

working tree에 새 파일을 생성하면 그 파일은 untracked 상태이다.

untracked 파일을 staging area에 등록하면 staged 상태가 된다.

staged 파일을 commit 하면 unmodified 상태가 된다.

unmodified 파일을 수정하면 modified 상태가 된다.

modified 파일을 staging area에 등록하면 staged 상태가 된다.

staged 파일을 commit 하면 unmodified 상태가 된다.

참고:

staged 되지 않은 파일은 commit 되지 않는다.

### staging area 등록 대상 파일

untracked 파일

새로 만든 파일은, staged 되고 commit 되어야 한다.

modified 파일

수정된 파일은, staged 되고 commit 되어야 한다.

### Lify cycle

(새 파일 만들기)

untracked

(버전 기록 대상으로 선택)

staged

(commit)

unmodified

(소스코드 수정)

modified

(버전 기록 대상으로 선택)

staged

(commit)

unmodified

(소스코드 수정)

modified

(버전 기록 대상으로 선택)

staged

(commit)

unmodified

## 2. git 기초 명령

### 1) local repository 명령

#### local repository 새로 생성하기

현재 디렉토리에 local repository 새로 생성하기

```
git init
```

#### working tree의 파일 상태 확인

```
git status
```

### 2) staging area 등록/취소 명령

#### 파일을 staging area에 등록하기

```
git add 파일명
```

예:

```
git add a.txt
```

현재 디렉토리의 a.txt 파일을 staging area에 등록한다.

```
git add *.txt
```

현재 디렉토리의 \*.txt 패턴의 파일들 중 staging 대상 파일들을 staging area에 등록한다.

```
git add "*.txt"
```

working tree의 \*.txt 패턴의 파일들 중 staging 대상 파일들을 staging area에 등록한다.  
(working tree => 현재 디렉토리 뿐만 아니라 아래 자식 디렉토리들 포함)

```
git add *
```

현재 디렉토리의 모든 staging 대상 파일들을 staging area에 등록한다.

```
git add .
```

working tree의 모든 staging 대상 파일들을 staging area에 등록한다.

#### staging area에 등록 취소하기

```
git reset 파일명
```

staging area에 등록 취소 명령이다

예:

```
git reset a.txt
```

a.txt 파일을 staging area에 등록을 취소한다.

```
git reset
```

staging area의 모든 파일을 등록 취소한다.

```
git rm --cached 파일명
```

staging area에 등록 취소 명령이다

예:

```
git rm --cached a.txt
```

a.txt 파일을 staging area에 등록을 취소한다.

### 3) commit 명령

#### commit 하기

```
git commit
```

staging area에 등록된 파일들이 local repository에 새 버전으로 등록된다.

위 명령을 실행하면, commit 메시지를 입력하기 위한 편집기가 자동으로 열린다.

commit 메시지를 입력하고 저장한 후 편집기를 닫으면, commit 명령이 완료된다.

저장하지 않고 편집기를 닫으면, commit 명령이 취소된다.

#### commit 목록 보기

```
git log
```

```
git log --stat
```

commit에 기록된 파일 목록과 삽입/삭제된 줄 수 보기

### 3. 실습

#### 1) Visual Studio Code 설치

개발 도구 설치 강의노트를 참고하라.

#### PATH 환경 변수 확인

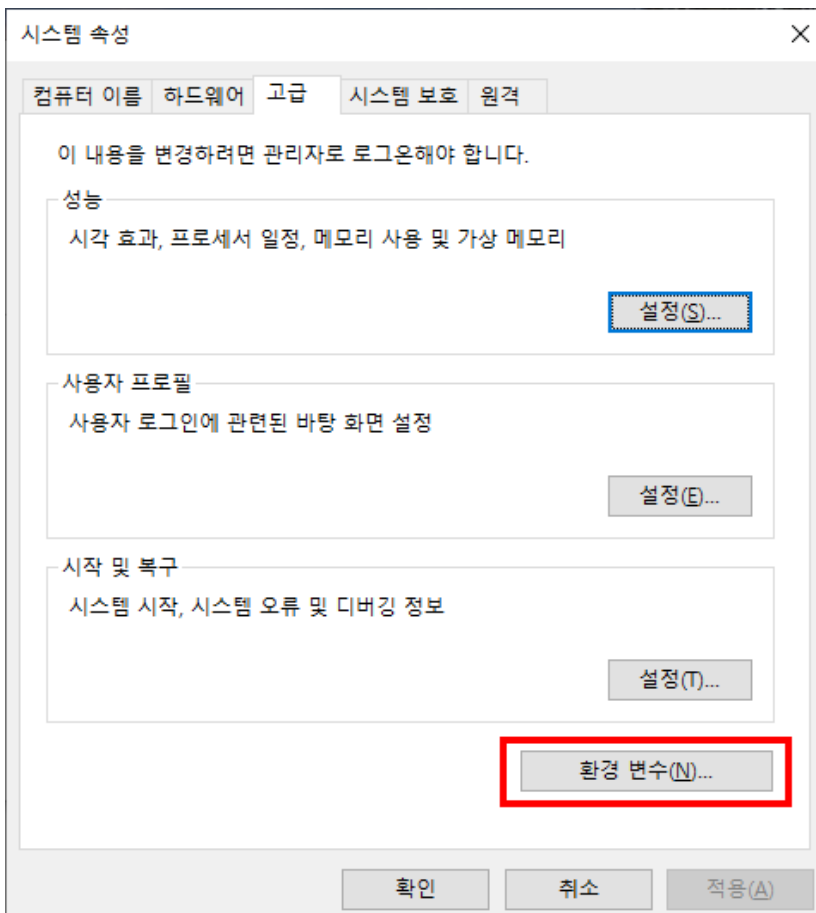
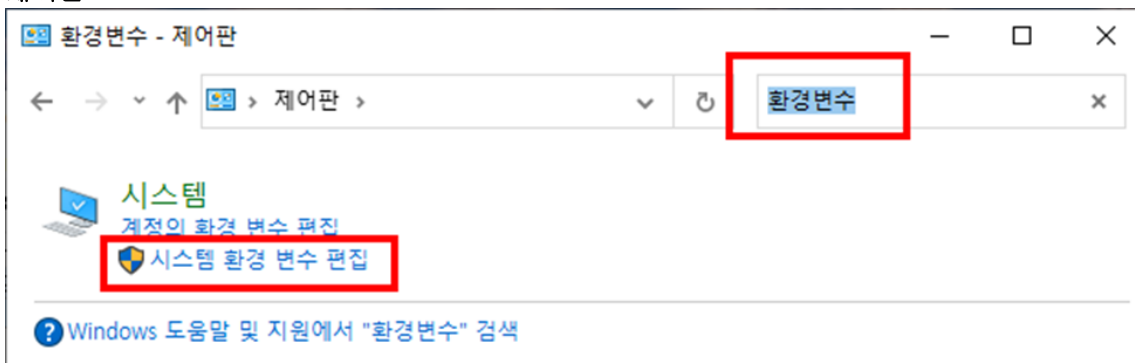
Visual Studio Code 디렉토리가 PATH 환경 변수에 등록되어 있다면,  
bash shell 이나, 명령 프롬프트에서 다음 명령을 실행하면, Visual Studio Code가 실행된다.

```
code
```

#### PATH 환경 변수 등록

Visual Studio Code 디렉토리가 PATH 환경 변수에 등록되어 있지 않다면, 등록하자.

제어판





## 환경 변수

### Seungjin에 대한 사용자 변수(U)

| 변수                | 값                                       |
|-------------------|-----------------------------------------|
| OneDriveCommer... | C:\D\OneDrive - 성공회대학교                  |
| OneDriveConsumer  | C:\D\OneDrive                           |
| Path              | C:\Users\Seungjin\AppData\Local\Micr... |
| TEMP              | C:\Users\Seungjin\AppData\Local\Temp    |

새로 만들기(N)...

편집(E)...

삭제(D)

### 시스템 변수(S)

| 변수               | 값                                      |
|------------------|----------------------------------------|
| ComSpec          | C:\WINDOWS\system32\cmd.exe            |
| DriverData       | C:\Windows\System32\Drivers\DriverData |
| NUMBER_OF_PRO... | 12                                     |
| OS               | Windows_NT                             |

새로 만들기(W)...

편집(I)...

삭제(L)

확인

취소

## 환경 변수 편집

C:\Users\Seungjin\AppData\Local\Microsoft\WindowsApps  
 C:\Java\zulu11.54.25-ca-jdk11.0.14.1-win\_x64\bin  
 C:\Users\Seungjin\AppData\Local\Programs\Microsoft VS Code\bin  
 C:\Users\Seungjin\dotnet\tools  
 C:\D\OneDrive - 성공회대학교\A\bin  
 C:\D\OneDrive - 성공회대학교\A\python  
 C:\Program Files\MySQL\MySQL Server 8.0\bin  
 C:\Users\Seungjin\AppData\Roaming\npm

새로 만들기(N)

편집(E)

찾아보기(B)...

삭제(D)

위로 이동(U)

아래로 이동(O)

텍스트 편집(T)...

확인

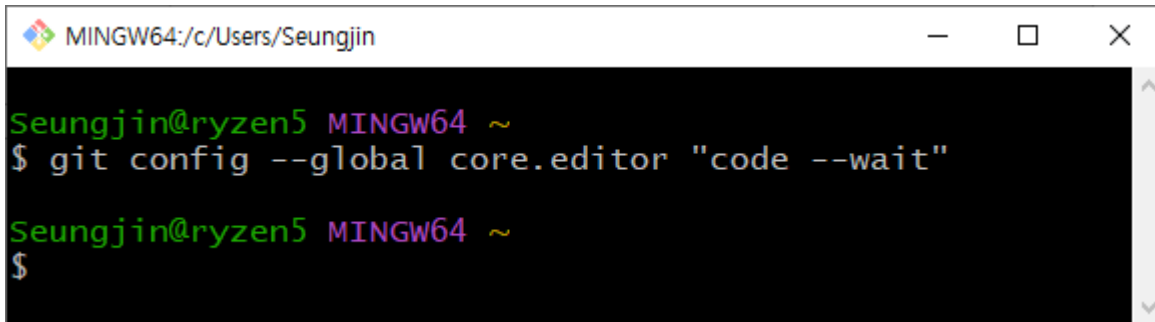
취소

만약 Visual Studio Code 항목이 PATH 환경변수 목록에 없다면,  
찾아보기 버튼을 클릭하여, 다음 폴더를 선택하여 등록해야 한다.

```
C:\Users\Seungjin\AppData\Local\Programs\Microsoft VS Code\bin
```

## 2) Visual Studio Code를 git 에디터로 등록

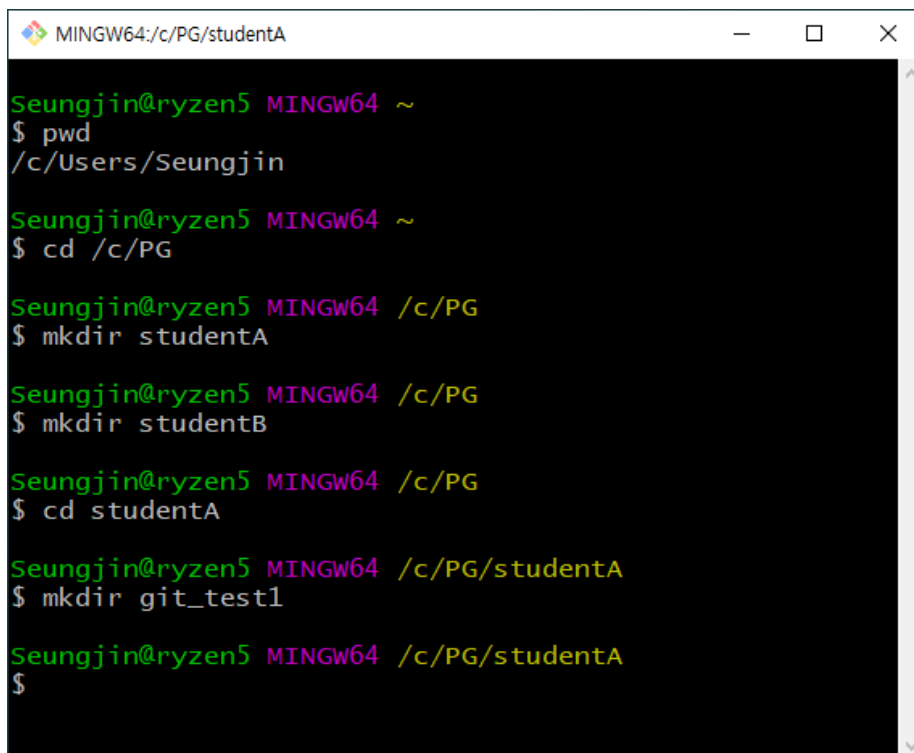
```
git config --global core.editor "code --wait"
```

A screenshot of a terminal window titled 'MINGW64:/c/Users/Seungjin'. The prompt is 'Seungjin@ryzen5 MINGW64 ~'. The command '\$ git config --global core.editor "code --wait"' has been entered and executed. The prompt has changed to 'Seungjin@ryzen5 MINGW64 ~' and a new '\$' prompt is visible on the next line.

```
MINGW64:/c/Users/Seungjin
Seungjin@ryzen5 MINGW64 ~
$ git config --global core.editor "code --wait"
Seungjin@ryzen5 MINGW64 ~
$
```

### 3) 프로젝트 폴더 만들기

적당한 위치에 프로젝트 폴더를 만들자.



```
MINGW64:/c/PG/studentA

Seungjin@ryzen5 MINGW64 ~
$ pwd
/c/Users/Seungjin

Seungjin@ryzen5 MINGW64 ~
$ cd /c/PG

Seungjin@ryzen5 MINGW64 /c/PG
$ mkdir studentA

Seungjin@ryzen5 MINGW64 /c/PG
$ mkdir studentB

Seungjin@ryzen5 MINGW64 /c/PG
$ cd studentA

Seungjin@ryzen5 MINGW64 /c/PG/studentA
$ mkdir git_test1

Seungjin@ryzen5 MINGW64 /c/PG/studentA
$
```

#### pwd

현재 디렉토리 경로명을 확인하는 명령이다.

git bash 셸을 처음 열었을 때 현재 디렉토리는 사용자 홈 디렉토리이다. (C:/Users/계정명)

#### cd /c/PG

c:/PG 디렉토리로 이동한다.

실습 하기에 적당한 폴더를 만들고, 그 폴더로 이동하자.

#### mkdir studentA

#### mkdir studentB

현재 디렉토리 아래에 studentA 디렉토리와 studentB 디렉토리를 만든다.

두 학생이 git으로 협업하는 것을 흉내를 내기 위해서, studentA, studentB 폴더를 만들자.

#### cd studentA

studentA 디렉토리로 이동한다.

#### mkdir git\_test1

현재 디렉토리 아래에 git\_test1 디렉토리를 만든다.

## 4) local repository 만들기 (git init)

```
MINGW64:/c/PG/studentA/git_test1

Seungjin@ryzen5 MINGW64 /c/PG/studentA
$ cd git_test1/

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1
$ git init
Initialized empty Git repository in C:/PG/studentA/git_test1/.git/

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ ls -al
total 4
drwxr-xr-x 1 Seungjin 197609 0 Sep  2 13:14 ./
drwxr-xr-x 1 Seungjin 197609 0 Sep  2 13:11 ../
drwxr-xr-x 1 Seungjin 197609 0 Sep  2 13:14 .git/

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$
```

### cd git\_test1

git\_test1 디렉토리로 이동한다.

### git init

현재 디렉토리에 local repository를 만든다. (git\_test1 디렉토리에)

### ls -al

현재 디렉토리의 파일 목록을 출력한다.

### local repository

.git 디렉토리가 생성되었다.

이 디렉토리가 현재 프로젝트의 local repository 이다.

## 5) Visual Studio Code 열기

git\_test1 프로젝트 디렉토리에서 Visual Studio Code를 열자.

```
code .
```

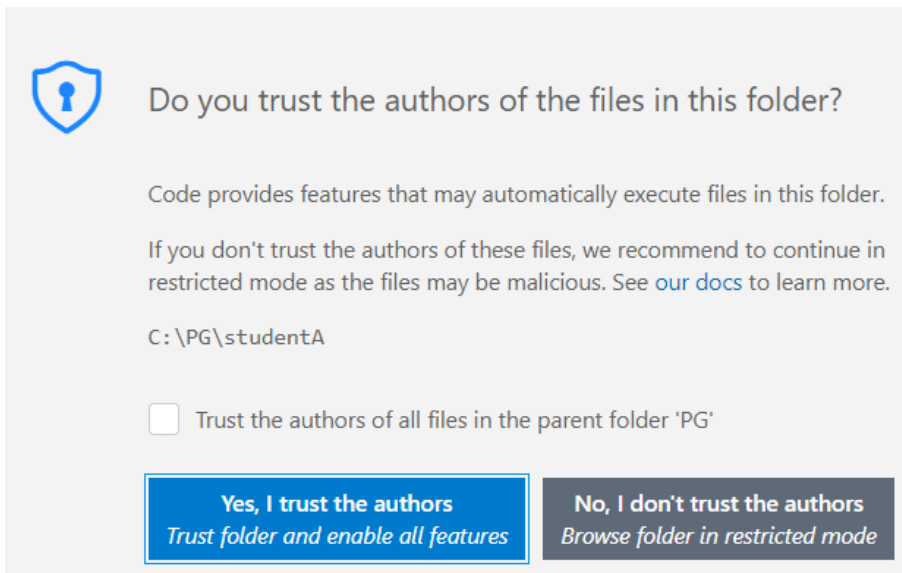
썸 문자를 입력해야 한다.

```
MINGW64:/c/PG/studentA/git_test1

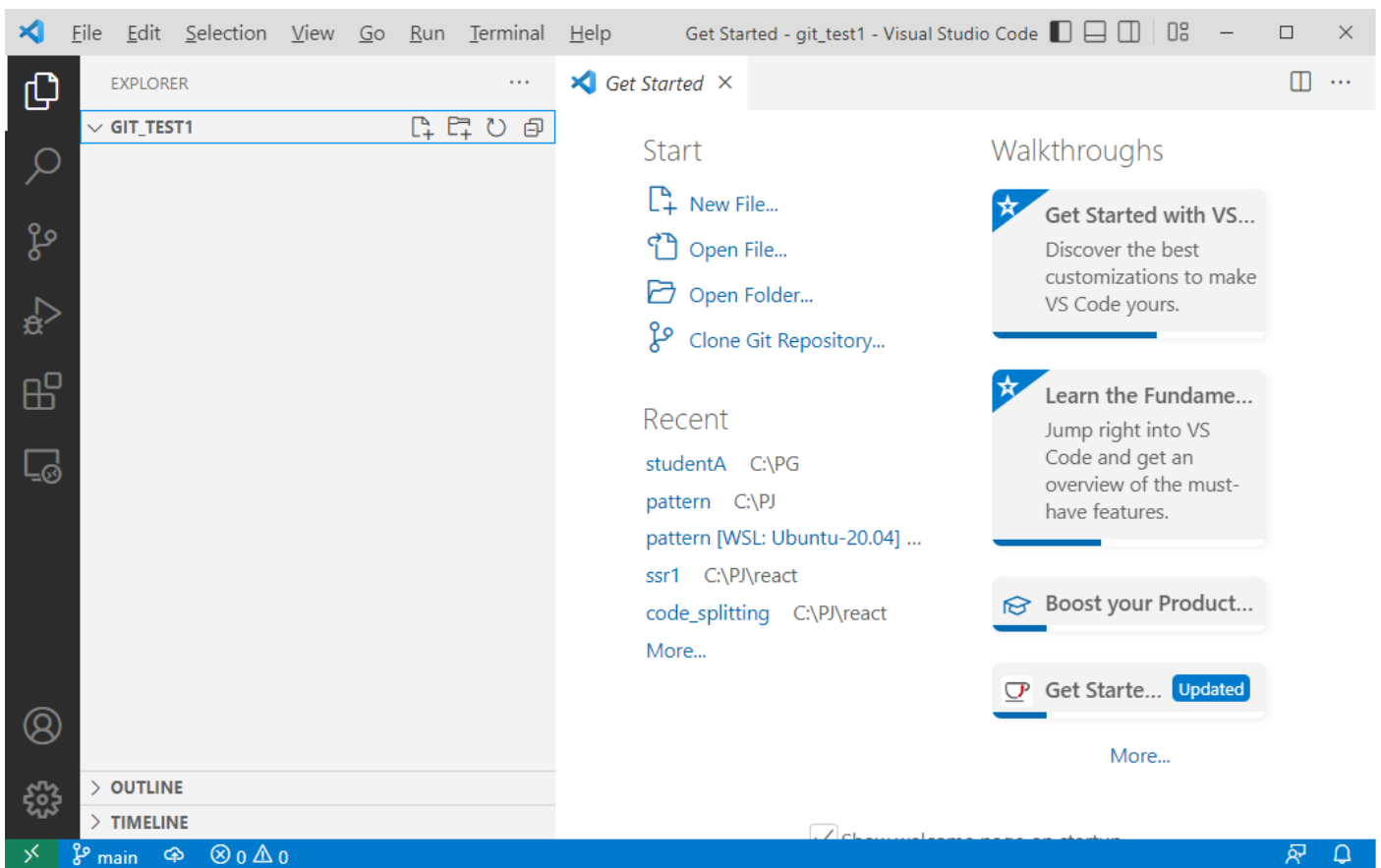
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1
$ git init
Initialized empty Git repository in C:/PG/studentA/git_test1/.git/

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ code .

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$
```



yes, I trust the authors  
 클릭



## 용어 정리

터미널(terminal), 명령 프롬프트, 셸(shell) 모두 비슷한 용어이다.

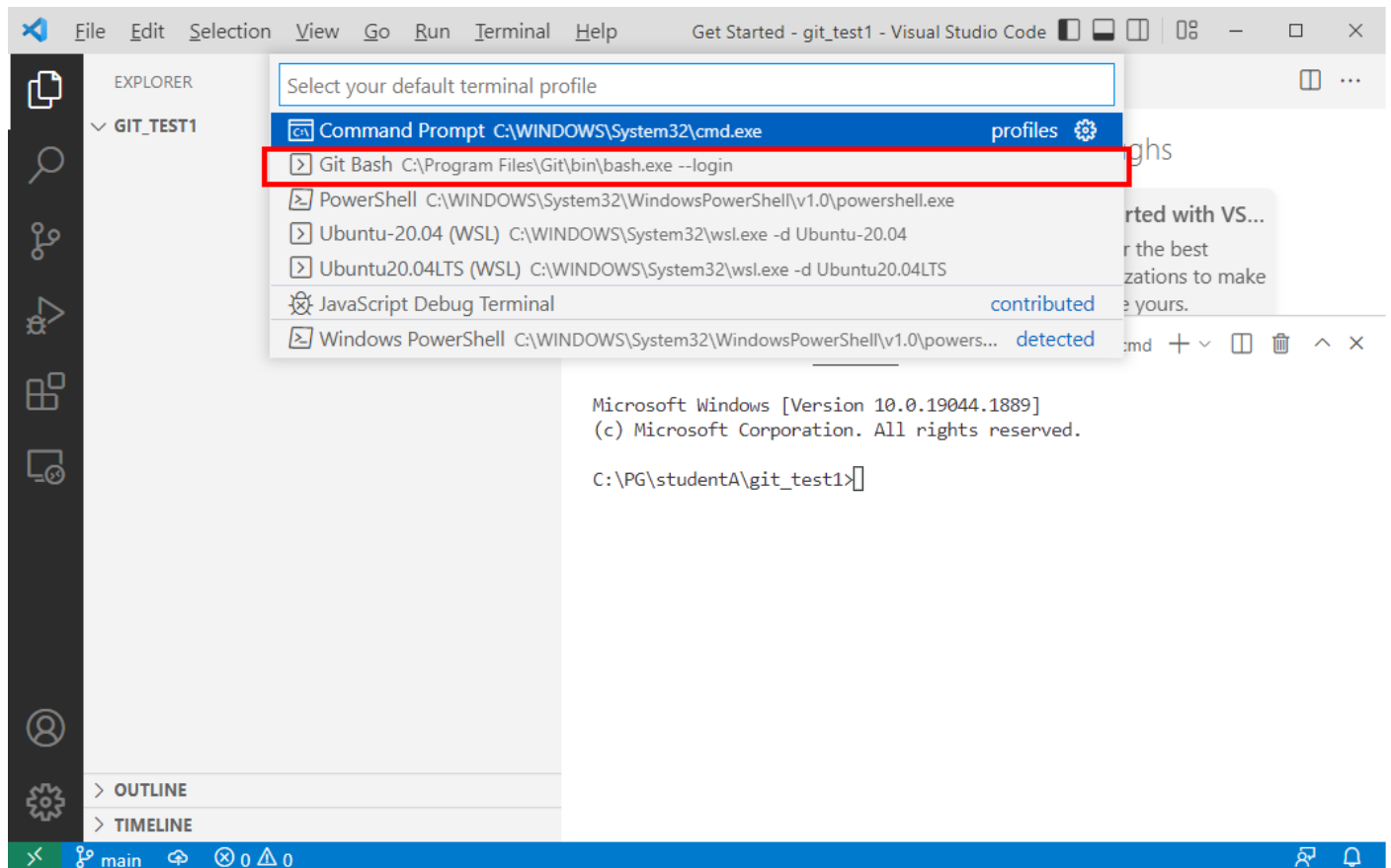
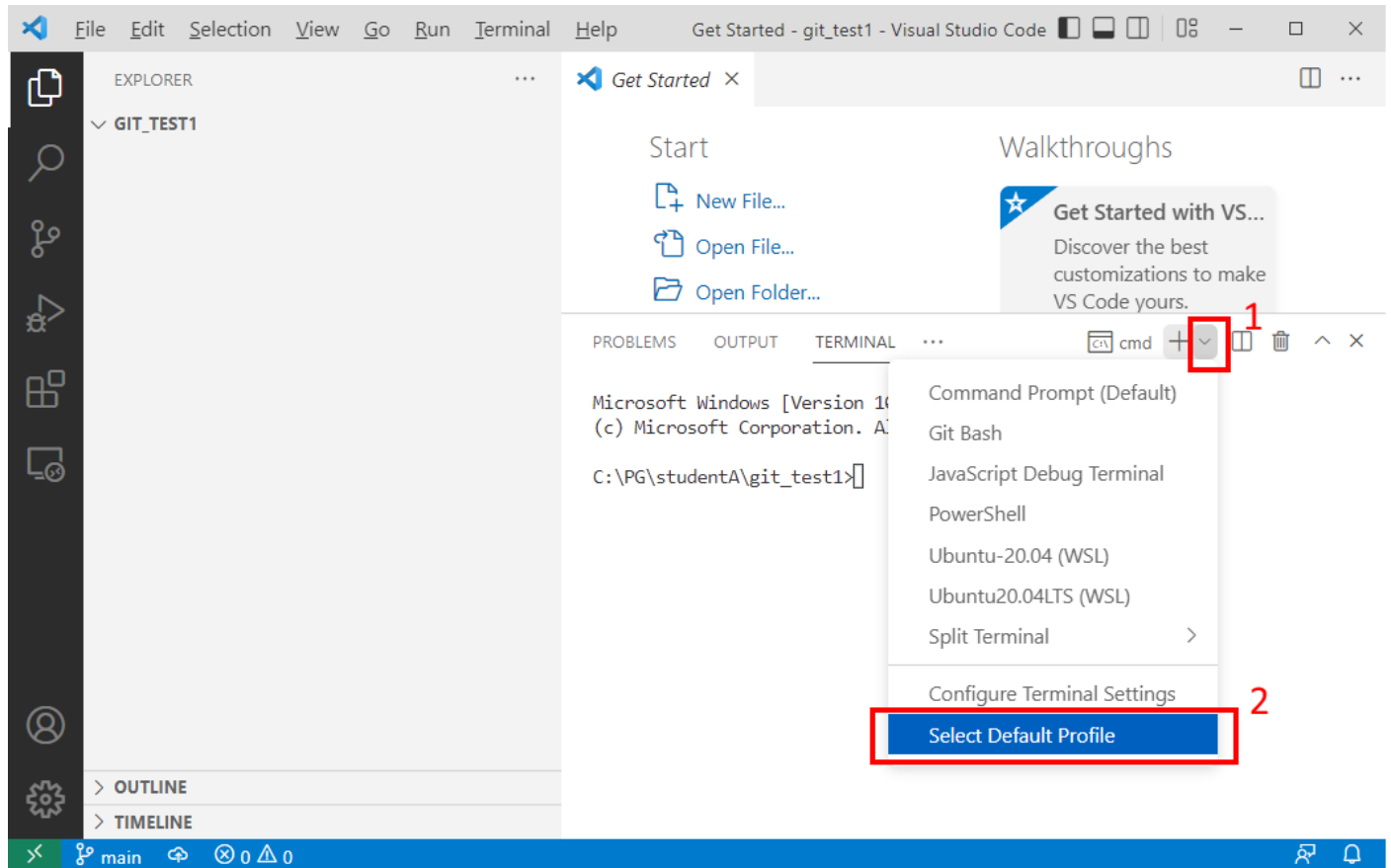
| Windows | 유닉스  | 설명                    |
|---------|------|-----------------------|
| 명령 프롬프트 | 터미널  | 명령 프롬프트 창, 터미널 창 그 자체 |
| cmd.exe | bash | 창 내부에서 실행되는 SW        |

cmd.exe, bash 같은 SW를 셸(shell) 이라고 한다.

## Visual Studio Code'에서 터미널 창 열기

단축키: Ctrl+~ 맥: Command+~

### 터미널 창에서 실행할 기본 셸 변경하기



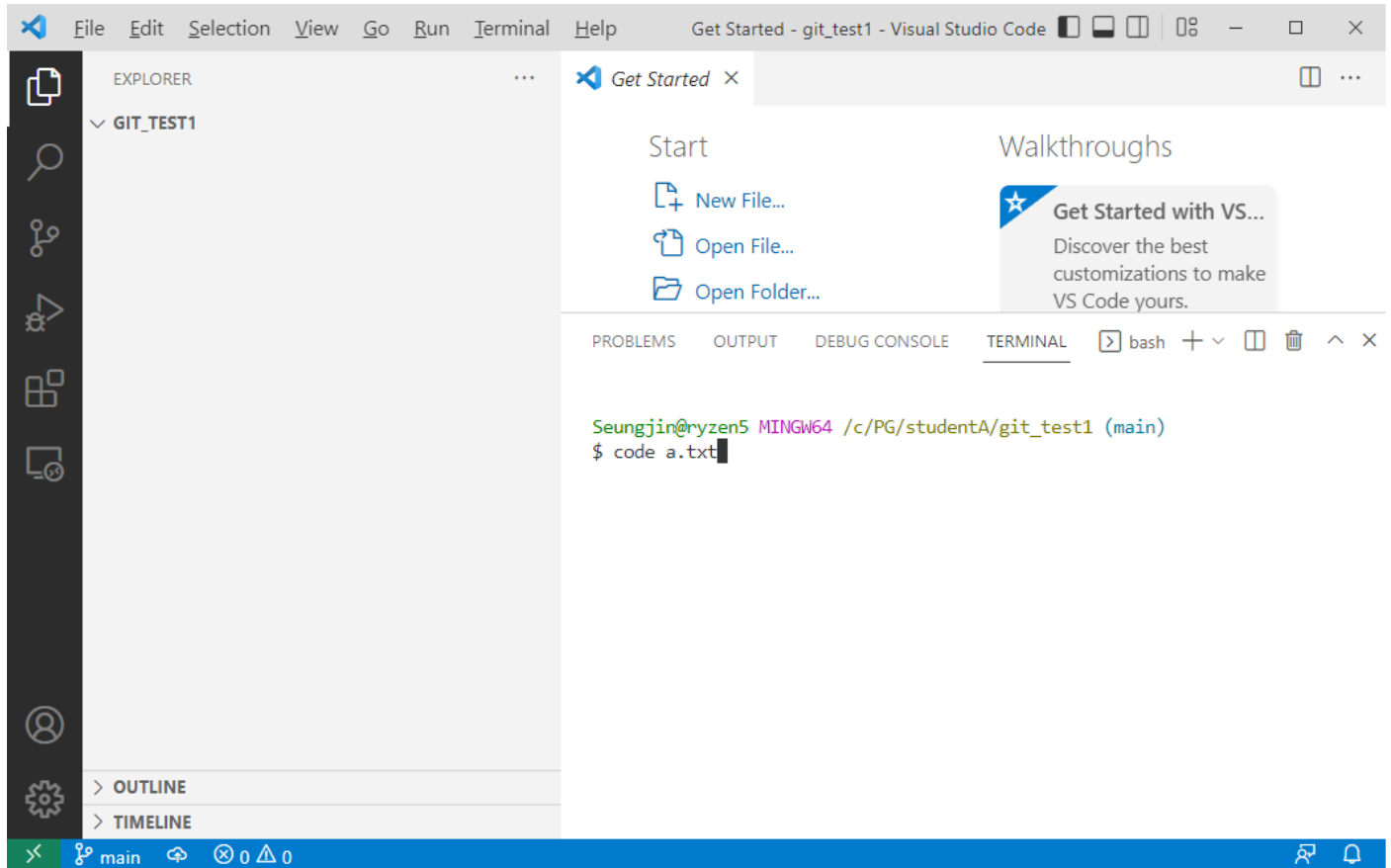
git bash 셸 선택

## 6) 소스코드 파일 만들기

현재 디렉토리에 소스코드 파일을 만들자.

```
code a.txt
```

git bash 쉘에서 위 명령을 실행하면,  
현재 디렉토리에 a.txt 파일을 만들기 위한 편집 창이,  
Visual Studio Code에서 열린다.



### a.txt

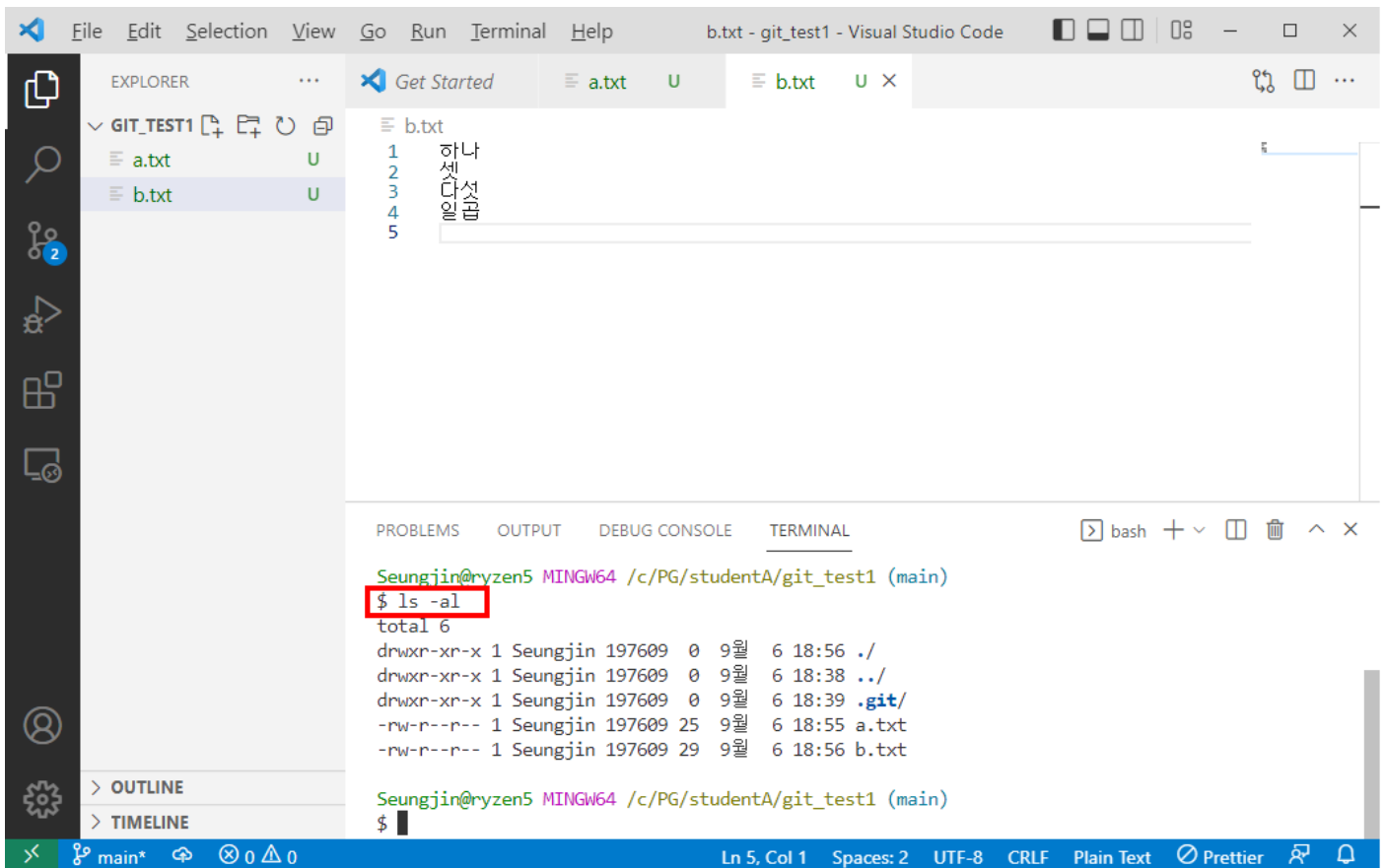
```
one  
three  
five  
seven
```

편집장에 위 내용을 입력하고, Ctrl+S 버튼을 눌러서 저장하자.  
현재 디렉토리에 a.txt 파일이 생성된다.

같은 방법으로 b.txt 파일을 생성하자.

### b.txt

```
하나  
셋  
다섯  
일곱
```



디렉토리 확인 명령 ls -al



## 7) 파일 상태 확인 (git status)

파일 상태 확인 명령은 다음과 같다.

```
git status
```

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        a.txt
        b.txt

nothing added to commit but untracked files present (use "git add" to track)

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

위 화면의 메시지 내용은 다음과 같다.

### No commits yet

아직 commit이 없음

### untracked files

a.txt

b.txt

위 두 파일은 untracked 상태이다.

### nothing added to commit but untracked files present

이 메시지의 뜻은 다음과 같다.

modified 상태의 파일이 없다 그러나 untracked 상태의 파일이 있다.

## 8) 새 파일을 tracked 상태로 바꾸기 (git add)

새 파일을 tracked 상태로 바꾸는 명령

```
git add 파일명
```

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git add a.txt b.txt
warning: in the working copy of 'a.txt', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'b.txt', CRLF will be replaced by LF the next time Git touches it

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   a.txt
        new file:   b.txt

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

위 화면의 메시지 내용은 다음과 같다.

**warning: CRLF will be replaced by LF in a.txt.**  
**The file will have its original line endings in your working directory**  
Windows 줄바꿈 문자 (CR LF) 두 문자가, Unix 줄바꿈 문자 (LF)로 변경되어 repository에 등록된다.  
그렇지만 working directory의 파일은 수정되지 않는다.

### On branch main

현재 작업 브랜치는 main 이다.

### No commits yet

아직 commit 없다.

### changes to be committed

수정된 파일 중에서 commit 대상은 다음과 같다.

## 9) 수정 내용 등록하기 (git commit)

파일 수정 내용을 local repository에 등록하는 명령

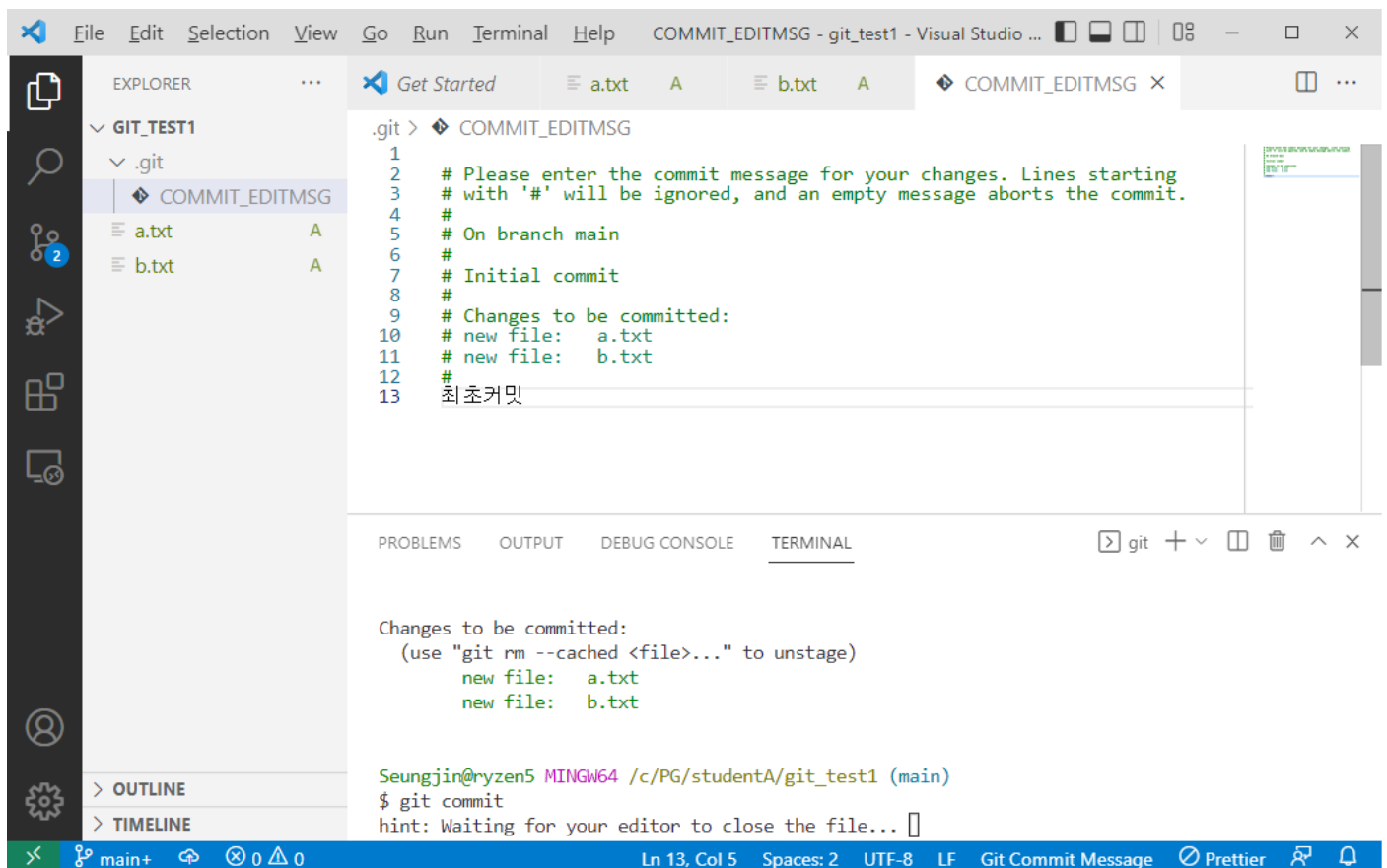
```
git commit
```

위 명령을 실행하면, commit 메시지를 입력하기 위한 편집기가 자동으로 열릴 것이다.  
git 클라이언트 설치할 때, 선택한 텍스트 편집기가 열린다.  
notepad++ 편집기를 추천한다.

메시지 입력

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
#
# Initial commit
#
# Changes to be committed:
#   new file:   a.txt
#   new file:   b.txt
#
최초 커밋
```

#으로 시작되는 라인은 메시지에 포함되지 않고 무시된다.



메시지를 입력하고, Ctrl+S 키를 눌러서 저장하고, 편집창을 닫으면  
git commit 명령이 완료된다.

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git commit
[main (root-commit) c70628d] 최초 커밋
 2 files changed, 8 insertions(+)
 create mode 100644 a.txt
 create mode 100644 b.txt

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

위 화면의 메시지 내용은 다음과 같다.

### **[main (root-commit) c70628d] 최초 커밋**

main 브랜치에 커밋 되었음.

커밋의 hash: c70628d

커밋 메시지: 최초 커밋

### **2 files changed, 8 insertions(+)**

수정된 파일은 두 개이고, 8 줄이 추가됨.

## 10) 커밋 목록 보기 (git log)

커밋 목록을 보는 명령

```
git log
```

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git log
commit c70628d42fefa8ffb467e94a3c5e30f9984c08a0 (HEAD -> main)
Author: Lee, Seungjin <quack337@gmail.com>
Date:   Tue Sep 6 18:58:35 2022 +0900
```

최초커밋

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

위 화면의 메시지 내용은 다음과 같다.

**commit c70628d42fefa8ffb467e94a3c5e30f9984c08a0**

commit hash는 f47b3ef..... 이다.

보통 hash의 앞 7자만 사용한다.

**HEAD -> main**

현재 작업하고 있는 브랜치의 선두 버전이, main 브랜치의 이 커밋이다.

## 4. remote repository 명령

### 1) remote repository에 연결

현재 디렉토리의 local repository를 remote repository에 연결하기 위한 명령은 다음과 같다.

```
git remote add 별칭 URL
```

별칭 => remote repository의 짧은 이름

URL => remote repository URL

예:

```
git remote add origin https://github.com/quack337/git_test1.git
```

위 명령에서 remote repository URL은 다음과 같다.

[https://github.com/quack337/git\\_test1.git](https://github.com/quack337/git_test1.git)

위 remote repository에 **origin** 이라는 별칭(별명)을 부여한다.

### 2) remote repository에 업로드

#### 최초 업로드

```
git push -u 원격저장소_별칭 업로드할_브랜치
```

local repository의 내용을 remote repository에 최초 업로드할 때는 위 명령을 실행한다.

원격저장소\_별칭: remote repository의 별칭을 말한다.

앞에서 remote repository URL을 연결할 때 사용한 별칭이다. (git remote add)

이 별칭은 **origin** 이다.

업로드할\_브랜치: 업로드할 브랜치 이름이다.

현재 작업 브랜치 이름은 **main** 이다.

예:

```
git push -u origin main
```

#### 최초 업로드 이후

최초 업로드 이후 또 업로드 할 때는 다음 명령을 실행하면 된다.

```
git push
```

### 3) remote repository에 다운로드

remote repository를 다운로드 하여 local repository를 새로 만드는 명령은 다음과 같다,  
local repository만 다운로드 하는 것이 아니고, working tree도 자동 생성됨.

```
git clone URL
```

예:

```
git clone https://github.com/quack337/git_test1.git
```

## 5. 실습

### 1) github에 계정 생성

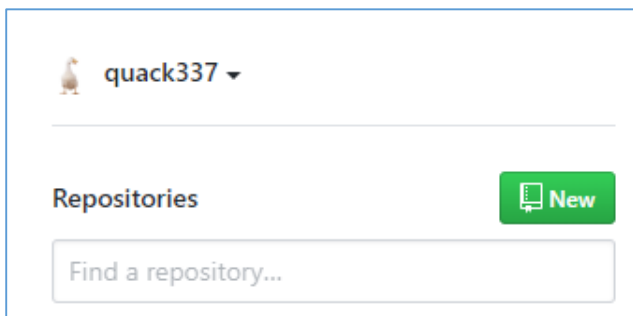
github 사이트에 회원 가입(sign up)을 하자.  
팀원 전부 회원 가입 해야 한다.  
gmail 주소로 가입하는 것을 추천한다.

<https://github.com/>

자세한 절차는 생략.

### 2) remote repository 생성

github 에 git\_test1 repository를 만들자.



New 클릭

### Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?  
[Import a repository.](#)

Owner \*

Repository name \*

quack337 / git\_test1 ✓

Great repository names are short and memorable. Need inspiration? How about [supreme-doodle?](#)

Description (optional)

git 명령 연습

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

☐ Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

☐ Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

Create repository 클릭

quack337 / git\_test1

Unwatch 1 Star 0 Fork 0

Code Issues Pull requests Actions Projects Wiki

**Quick setup — if you've done this kind of thing before** 가

Set up in Desktop or HTTPS SSH `https://github.com/quack337/git_test1.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

**...or create a new repository on the command line**

```
echo "# git_test1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/quack337/git_test1.git
git push -u origin main
```

**...or push an existing repository from the command line** 나

```
git remote add origin https://github.com/quack337/git_test1.git
git branch -M main
git push -u origin main
```

## 가) `https://github.com/quack337/git_test1.git`

생성된 remote repository의 URL 이다.

## 나)

```
git remote add origin https://github.com/quack337/git_test1.git
git branch -M main
git push -u origin main
```

local repository를 remote repository에 연결하고 최초 업로드하기 위한 git 명령이다.

```
git branch -M main
```

이 명령은, local repository의 git 기본 브랜치 이름이 main 이 아니고 master 인 경우에 이 이름을 main 으로 변경하는 명령이다.

그렇지만 우리는 Windows git 클라이언트를 설치할 때, 기본 브랜치 이름으로 main을 선택했기 때문에, 기본 브랜치 이름이 main 이다. 따라서 이 명령은 생략해도 된다.



### 3) remote repository에 연결

현재 디렉토리의 local repository를 remote repository에 연결하기 위한 명령은 다음과 같다.

```
git remote add origin https://github.com/quack337/git_test1.git
```

위 명령에서 remote repository URL은 다음과 같다.

[https://github.com/quack337/git\\_test1.git](https://github.com/quack337/git_test1.git)

위 remote repository에 **origin** 이라는 별칭(별명)을 부여한다.

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git remote add origin https://github.com/quack337/git_test1.git

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git remote -v
origin https://github.com/quack337/git_test1.git (fetch)
origin https://github.com/quack337/git_test1.git (push)

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

```
git remote -v
```

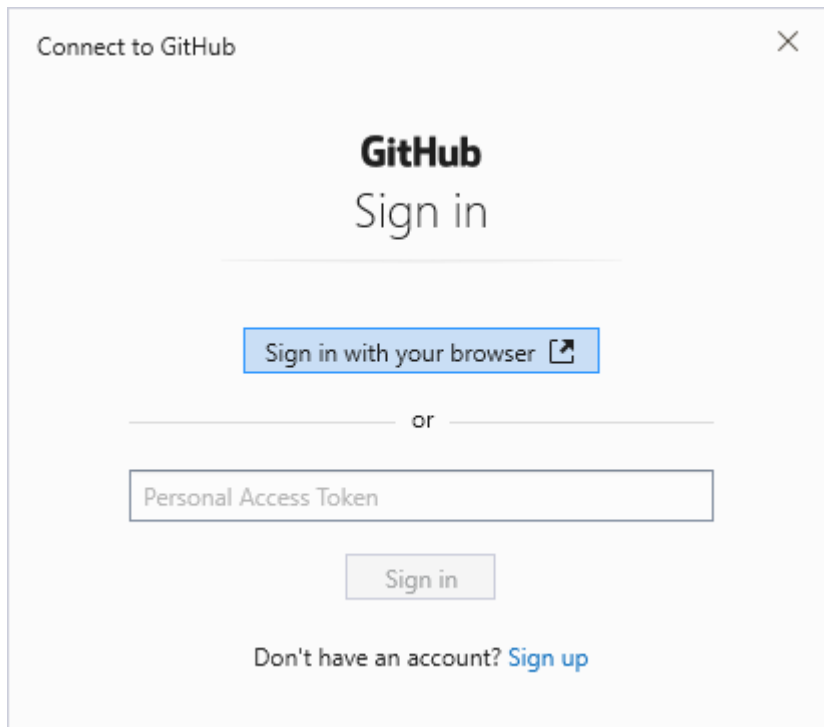
연결된 remote repository URL 확인

## 4) 작업 내용 업로드

### 최초 업로드

```
git push -u origin main
```

github에 처음 push 하는 경우에, 아래와 같은 창이 출력된다.



Sign in with your browser 버튼을 클릭하고  
github 에 로그인하자.

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 310 bytes | 310.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/quack337/git_test1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

## 최초 업로드 이후

최초 업로드 이후에는 또 업로드할 때는 다음 명령을 실행하면 된다.

```
git push
```

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ git push
Everything up-to-date

Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ █
```

이전 업로드 이후 commit 된 내용이 없으면, 업로드할 것도 없으므로,  
위와 같이 화면이 출력된다.

## 5) remote repository 다운로드

새로 팀에 참가하는 팀원(studentB)입장에서, remote repository를 처음 다운로드 하는 작업이다.

### studentB 디렉토리로 이동

cd 명령으로 studentB 디렉토리로 이동하자.

```
cd ../../studentB
```

### remote repository 다운로드 (clone)

remote repository를 다운로드 하여 local repository를 만드는 명령은 다음과 같다.

```
git clone URL
```

URL 부분에 remote repository URL을 입력해야 한다.

```
Seungjin@ryzen5 MINGW64 /c/PG/studentA/git_test1 (main)
$ cd ../../studentB

Seungjin@ryzen5 MINGW64 /c/PG/studentB
$ git clone https://github.com/quack337/git_test1.git
Cloning into 'git_test1'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), done.

Seungjin@ryzen5 MINGW64 /c/PG/studentB
$ █
```

```
git clone https://github.com/quack337/git_test1.git
```

위 명령에 의해

- (1) 현재 디렉토리 아래에 git\_test1 디렉토리가 생성되고
- (2) git\_test1 remote repository 가 다운로드 되어
- (3) git\_test1/.git local repository가 생성되고
- (4) git\_test1 디렉토리 아래에 working tree가 생성되었다.

```
Seungjin@ryzen5 MINGW64 /c/PG/studentB
$ cd git_test1/

Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test1 (main)
$ ls -al
total 6
drwxr-xr-x 1 Seungjin 197609  0  9월  6 19:07 ./
drwxr-xr-x 1 Seungjin 197609  0  9월  6 19:07 ../
drwxr-xr-x 1 Seungjin 197609  0  9월  6 19:07 .git/
-rw-r--r-- 1 Seungjin 197609 21  9월  6 19:07 a.txt
-rw-r--r-- 1 Seungjin 197609 25  9월  6 19:07 b.txt

Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test1 (main)
$ █
```

현재 디렉토리 아래에 git\_test1 디렉토리가 생성되었고  
이 디렉토리에 .git local repository 가 생성되었고  
소스 코드 파일도 전부 다운로드 되었음을 확인할 수 있다.

즉 studentA 가 업로드한 작업 공간이 전부 studentB 에게 복제되었다.

## 6. 과제

위 실습 절차를 그대로 따라서 수행하고,  
그 과정에서 생성된 git\_test1 remote repository URL을 제출하라.

예: [https://github.com/quack337/git\\_test1.git](https://github.com/quack337/git_test1.git)  
github remote repository URL은 .git 으로 끝난다.